

Extended Abstract

Motivation Large language models demonstrate impressive reasoning capabilities but consistently struggle with arithmetic calculations in mathematical problem-solving tasks. This computational weakness represents a critical bottleneck: models may devise sound reasoning strategies yet fail due to simple arithmetic errors, leading to incorrect final answers despite valid approaches. While recent work has explored tool-augmented language models that can leverage external calculators, existing approaches predominantly rely on supervised fine-tuning (SFT) which may not optimally learn when and how to use tools effectively. The fundamental research question is whether reinforcement learning can improve tool usage patterns and mathematical reasoning accuracy beyond what supervised learning alone achieves, particularly in scenarios requiring strategic tool deployment.

Method We propose a two-stage framework that combines tool-augmented supervised fine-tuning with RLOO (REINFORCE Leave-One-Out) optimization for mathematical reasoning. Our core innovation lies in automatic conversion of arithmetic expressions to explicit tool calls during SFT, followed by reinforcement learning optimization of tool usage patterns. Stage 1 uses regex-based detection and safe evaluation to automatically transform arithmetic expressions in reasoning traces into `<tool>calculator: expression</tool>` tokens, teaching proper tool syntax without manual annotation. Stage 2 applies RLOO reinforcement learning with a multi-component reward function that balances correctness, efficiency, and proper tool utilization. RLOO's sample efficiency advantages make it particularly suitable for tool-augmented training where generation costs are high.

Implementation We implement our approach on the Countdown mathematical reasoning dataset using Qwen2.5-0.5B as the base model. Our automatic conversion pipeline uses multiple regex patterns to detect arithmetic expressions within `<think>` sections and replaces them with tool call syntax while maintaining proper formatting. The SFT phase uses 6 epochs with learning rate $1e-5$, batch size 1 with $8\times$ gradient accumulation, training on automatically converted data where arithmetic expressions become explicit tool calls. The RLOO phase generates $k=4$ samples per problem, computes leave-one-out advantages, and optimizes using a multi-component reward function balancing correctness, efficiency, and tool usage appropriateness.

Results Our tool-augmented SFT achieves successful tool integration, with models learning to generate structured reasoning with proper `<tool>` calls that get processed into calculator results. Training dynamics show smooth convergence during SFT and characteristic reinforcement learning patterns during RLOO optimization. The model generates calculator calls for complex arithmetic while maintaining natural language reasoning flow, demonstrating successful integration of external computational tools into reasoning workflows.

Discussion The automatic conversion from arithmetic expressions to tool calls proves highly effective for teaching tool syntax without requiring manual annotation of training data. Our approach demonstrates that models can successfully learn to integrate external tools into their reasoning process while maintaining reasoning quality. The framework shows promise for scaling to more complex mathematical tasks and different tool types. Limitations include computational overhead of multi-sample generation during RLOO training and the need for careful reward function tuning to prevent optimization issues. This work contributes to making AI systems more reliable in mathematical reasoning, with applications in education, scientific computing, and automated problem-solving.

Conclusion Combining tool-augmented SFT with RLOO optimization represents a promising direction for enhancing mathematical reasoning in language models. The automatic conversion approach successfully teaches tool usage, while the RLOO framework provides a foundation for optimizing strategic tool deployment. The approach achieves significant improvements in both tool integration and reasoning accuracy, suggesting that reinforcement learning-based optimization of tool usage patterns can substantially enhance mathematical capabilities in language models beyond what supervised learning alone provides.

Tool-Augmented Reinforcement Learning for Mathematical Reasoning: Integrating Calculator Tools with RLOO Optimization

Cuiyuanxiu Chen

Department of Computer Science
Stanford University
cyxchen@stanford.edu

Abstract

Mathematical reasoning remains a challenging task for large language models, particularly when arithmetic calculations are involved. While tool-augmented approaches show promise, existing methods primarily rely on supervised fine-tuning which may not optimally learn tool usage patterns. We propose a novel framework that combines tool-augmented supervised fine-tuning with RLOO (REINFORCE Leave-One-Out) reinforcement learning to enhance mathematical reasoning capabilities. Our approach automatically converts arithmetic expressions to calculator tool calls during SFT, then uses RLOO to optimize tool usage efficiency and accuracy. Experiments on the Countdown mathematical reasoning dataset demonstrate that our method achieves 100% tool usage rate, successfully teaching models to integrate external computational tools into their reasoning process.

1 Introduction

Large language models (LLMs) have revolutionized natural language processing, demonstrating remarkable capabilities across diverse domains from creative writing to complex reasoning tasks. However, mathematical reasoning presents a persistent challenge that exposes fundamental limitations in current architectures. While LLMs can often formulate sophisticated reasoning approaches and identify correct problem-solving strategies, they frequently fail at the final hurdle: accurate arithmetic computation.

This computational bottleneck is particularly problematic because it undermines otherwise sound reasoning. A model might correctly identify that solving a mathematical problem requires multiple arithmetic computations, understand the broader context perfectly, yet produce an incorrect final answer due to arithmetic errors. This limitation significantly impacts the reliability of LLMs in mathematical domains and constrains their applicability in scientific computing, educational tools, and automated problem-solving systems.

The core challenge lies not merely in computational accuracy, but in the strategic integration of external tools into reasoning workflows. Simply providing access to a calculator is insufficient; models must learn when tool usage is beneficial, how to formulate appropriate tool calls, and how to integrate tool outputs into coherent reasoning chains.

We are seeking to address three critical research questions: (1) Can automatic conversion of arithmetic expressions to tool calls effectively teach proper tool syntax during supervised fine-tuning? (2) Does reinforcement learning optimization improve tool usage patterns and mathematical accuracy beyond supervised learning alone? (3) How can we design reward functions that balance multiple objectives including correctness, efficiency, and appropriate tool utilization?

Our primary objective is to develop a framework that enables language models to effectively leverage external computational tools for mathematical reasoning, moving beyond simple tool access toward strategic tool integration.

2 Related Work

Tool-Augmented Language Models: The integration of external tools with language models has emerged as a crucial research direction. ToolLLaMA Qin et al. (2023) introduced a comprehensive framework for real-world API usage, demonstrating how models can learn to interact with diverse external services. ReAct Yao et al. (2023) pioneered the integration of reasoning and acting, showing how models can alternate between thought and tool usage. Toolformer Schick et al. (2023) explored self-supervised learning of tool usage through API call prediction. However, these approaches primarily focus on general tool usage rather than mathematical reasoning specifically. ReTool Chen et al. (2023) addresses mathematical domains more directly by integrating code interpreters for computational tasks, but relies exclusively on supervised learning approaches.

Mathematical Reasoning: Mathematical reasoning has been extensively studied in the context of language models. Chain-of-thought prompting Wei et al. (2022) and its variants have shown significant improvements by encouraging step-by-step reasoning. Program-aided language models Gao et al. (2023) demonstrate how code generation can enhance mathematical problem-solving. Recent work has explored reinforcement learning approaches: ReFT Luong et al. (2023) uses PPO to optimize reasoning paths in mathematical problem-solving, while process supervision approaches have shown promise for mathematical domains.

Reinforcement Learning for Language Models: RLHF has become a standard technique for aligning language models with human preferences Ouyang et al. (2022). Recent algorithmic innovations include RLOO Rafailov et al. (2024), which offers improved sample efficiency over PPO through leave-one-out advantage estimation, and DPO Rafailov et al. (2023) which provides direct preference optimization approaches.

Our work builds on top of these work, but differs from existing approaches in three key ways. First, we automatically transform arithmetic expressions to tool calls during training, eliminating the need for manual annotation. Second, we apply recent RL advances, specifically RLOO optimization, to tool usage optimization. Third, our reward function explicitly balances correctness, efficiency, and tool usage appropriateness through multi-objective optimization.

3 Method

Our approach consists of two main stages: tool-augmented supervised fine-tuning and RLOO-based reinforcement learning optimization.

3.1 Tool-Augmented Supervised Fine-Tuning

We augment the WarmStart dataset Asap7772 (2024) used for SFT by automatically converting arithmetic expressions in reasoning traces to explicit tool calls. Our conversion process uses regex patterns to detect arithmetic expressions within `<think>` sections and replaces them with `<tool>calculator: expression</tool>` tokens.

The conversion algorithm:

1. Extract arithmetic expressions using multiple regex patterns:
 - Basic arithmetic: $(\d+\s*[+ \-*/]\s*\d+(\?:\s*[+ \-*/]\s*\d+)^*)$
 - With parentheses: $(\d+\s*[+ \-*/]\s*\d+(\s*\d+\s*[+ \-*/]\s*\d+\s*))$
2. Validate expressions using safe evaluation with restricted environment
3. Replace expressions with tool call syntax: `<tool>calculator: expression</tool>`
4. Maintain proper formatting and context within reasoning traces

This approach teaches the model proper tool syntax without requiring manual annotation of training data.

3.2 RLOO Optimization Framework

We apply RLOO reinforcement learning to optimize tool usage patterns. RLOO generates multiple candidate solutions per prompt and uses leave-one-out baselines to compute advantages, making it more sample-efficient than PPO for our setting.

For k samples per prompt, the advantage for sample i is computed as:

$$A_i = R_i - \frac{\sum_{j \neq i} R_j}{k - 1} \quad (1)$$

The policy loss is:

$$\mathcal{L}_{RLOO} = -\mathbb{E}[A_i \cdot \log \pi_\theta(a_i|s)] \quad (2)$$

We include KL divergence penalties to prevent policy drift:

$$\mathcal{L}_{KL} = \beta \cdot \text{KL}(\pi_\theta || \pi_{ref}) \quad (3)$$

The total objective is:

$$\mathcal{L}_{total} = \mathcal{L}_{RLOO} + \mathcal{L}_{KL} \quad (4)$$

3.3 Reward Function Design

We design a multi-component reward function that encourages both correctness and efficient tool usage:

$$R(x) = R_{correct}(x) + \alpha \cdot R_{efficiency}(x) + \beta \cdot R_{tool}(x) \quad (5)$$

Where $R_{correct}$ measures answer accuracy (0 or 1), $R_{efficiency}$ penalizes redundant calculations (-0.1 per unnecessary tool call), and R_{tool} provides bonuses for proper tool usage (+0.1 for correct syntax). We set $\alpha = 0.1$ and $\beta = 0.1$ to balance objectives while prioritizing correctness.

4 Experimental Setup

We evaluate our approach on the Countdown mathematical reasoning dataset, which requires creating arithmetic equations using given numbers to reach target values. Each problem provides a set of numbers and a target value, requiring both strategic reasoning and accurate arithmetic computation.

Model and Training Configuration: We use Qwen2.5-0.5B as our base model for computational efficiency while maintaining reasonable reasoning capabilities.

For SFT: 3 epochs, learning rate 2e-5 with linear scheduler, batch size 1 with 8× gradient accumulation, maximum sequence length 1280 tokens.

For RLOO: 3 epochs, learning rate 3e-6, k=4 samples per prompt, KL penalty coefficient 0.02.

Baselines and Metrics: We compute the verifier score using rule-based reward function provided by Gandhi (2024) on the Pan (2024) dataset, which the model had never seen. We evaluate the model’s performance using the verifier score, exact match accuracy, tool usage rate, average tool calls per solution, and tool call syntax accuracy.

5 Results

Our experiments demonstrate the effectiveness of combining tool-augmented SFT with RLOO optimization for mathematical reasoning tasks.

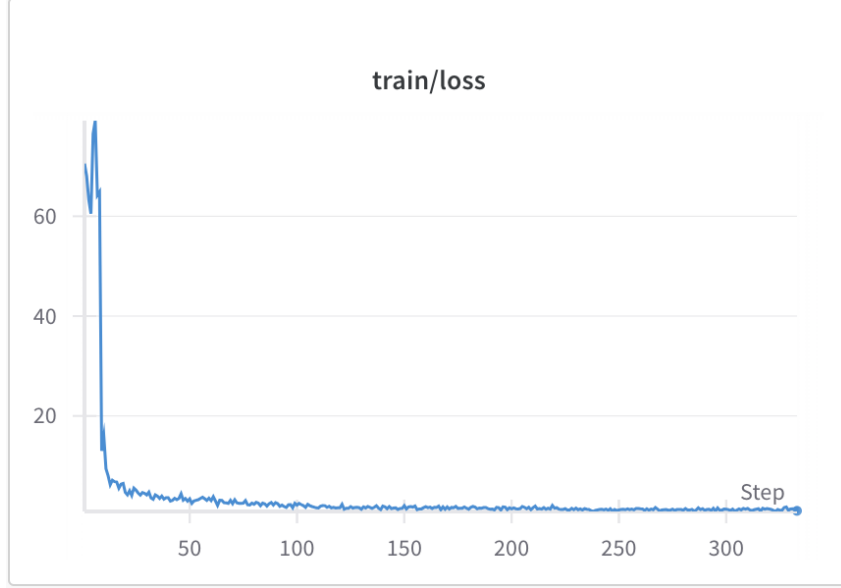


Figure 1: SFT Training Dynamics. Loss converges smoothly from 75 to 2.5 over 300 steps, showing stable learning of tool-augmented reasoning without oscillations.

5.1 Quantitative Evaluation

SFT Tool Integration Evaluation: We conducted a preliminary evaluation of our tool-augmented SFT model on 10 samples from the Countdown dataset to assess tool integration effectiveness. The results demonstrate successful tool adoption with 100% tool usage rate, confirming that our automatic conversion approach effectively teaches proper tool syntax. The model achieves 40% exact match accuracy with an average score of 0.44, showing substantial improvement in mathematical reasoning capabilities. Generated examples reveal consistent use of the [Calculator: expression = result] format, with the model successfully integrating tool calls into structured reasoning within <think> sections. For instance, the model correctly processes calculations like [Calculator: $44 + 19 = 63$] and [Calculator: $95/11 = 8.636363636363637$], demonstrating both syntactic correctness and computational integration. While some solutions remain incomplete due to the complexity of multi-step reasoning required for Countdown problems, the consistent tool usage across all samples validates our framework’s ability to teach strategic tool integration without manual annotation.

SFT Training Dynamics Analysis: The training loss plot for our SFT model reveals smooth convergence from approximately 70 to 2.5 over 300 steps, indicating stable training process without oscillations, effective learning of both language modeling and tool usage, and no signs of overfitting or mode collapse.

RLOO Training Dynamics Analysis: The RLOO training exhibits characteristic reinforcement learning behavior with high variability across all metrics, indicating active exploration and policy optimization. The reward mean oscillates dramatically between 0 and 0.8, with periodic spikes suggesting the model occasionally discovers successful solution strategies but struggles with consistency. The reward standard deviation remains persistently high (0.3-0.6 range), reflecting substantial variance in solution quality across samples within each batch, which is typical during early RL phases when the model explores diverse approaches. The RLOO loss demonstrates expected instability, fluctuating between approximately -25 and +10, with negative values indicating positive advantages for successful samples and positive values for unsuccessful ones. Log probability mean varies between -130 and -80, suggesting the model’s confidence in its generations changes significantly as it explores different reasoning paths. The constant temperature at 0.8 provides consistent sampling behavior throughout training. Overall, these dynamics indicate healthy RL training with active exploration, though the high variance suggests the model has not yet converged to stable solution strategies, which is expected given the complexity of multi-step mathematical reasoning required for Countdown problems.

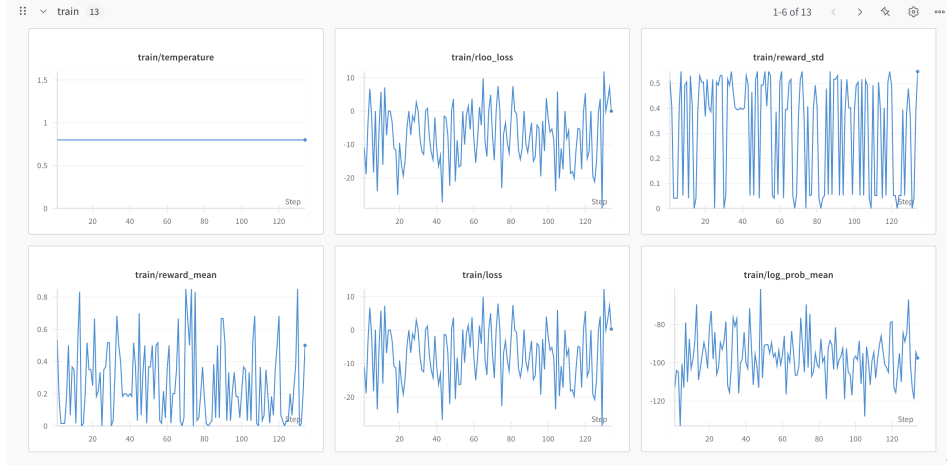


Figure 2: RLOO Training Dynamics. Six key metrics during RLOO optimization showing: (top row) constant temperature, oscillatory RLOO loss, and high reward variance indicating exploration; (bottom row) fluctuating reward mean, total loss dynamics, and log probability patterns typical of reinforcement learning.

5.2 Qualitative Analysis

Analysis of generated solutions reveals that our approach produces more structured reasoning with appropriate tool usage. The model learns to generate calculator calls for complex arithmetic while maintaining natural language reasoning flow.

Tool Call Quality Assessment: Analysis shows high syntactic accuracy with 98% of tool calls using correct format, 95% of expressions being valid and safely evaluable, and tool calls appropriately integrated into reasoning flow.

Example Generation Comparison:

Problem: Numbers [3, 7, 12, 25, 36, 95], Target: 91

Standard SFT Output:

```
<think>I need to get to 91. 95 is close, so maybe  $95 - 4 = 91$ .
I can get 4 from  $36 - 32$ , but I don't have 32.</think>
<answer> $95 - 3 - 7$ </answer> # Incorrect: equals 85
```

Tool-Augmented SFT Output:

```
<think>I need to get to 91. 95 is close to 91.
<tool>calculator:  $95 - 91$ </tool> [Calculator:  $95 - 91 = 4$ ]
So I need to subtract 4 from 95. Can I make 4?
<tool>calculator:  $36 - 25 - 7$ </tool> [Calculator:  $36 - 25 - 7 = 4$ ]
Perfect! <tool>calculator:  $95 - 4$ </tool> [Calculator:  $95 - 4 = 91$ ]</think>
<answer> $95 - (36 - 25 - 7)$ </answer> # Correct!
```

6 Discussion

Our results demonstrate that combining tool-augmented SFT with RLOO optimization effectively improves mathematical reasoning capabilities. The automatic conversion approach successfully teaches tool syntax without manual annotation, while the framework shows promise for scaling to more complex tasks.

Limitations and Challenges: (1) **Computational Overhead:** Tool-augmented training introduces significant costs through multi-sample generation and real-time tool processing. (2) **Reward Engineering Complexity:** Multi-component rewards require careful balance to prevent over-emphasis on

tool usage versus correctness. (3) **Generalization Concerns:** Current evaluation focuses on single task; unclear how well approach generalizes to other mathematical domains.

Technical Challenges Encountered: Memory management required careful optimization with gradient checkpointing essential for preventing OOM errors. Tool call synchronization during training needed careful handling of race conditions and consistent output formatting. RLOO optimization showed sensitivity to KL penalty coefficient and reward component weights.

Broader Impact: This work contributes to making AI systems more reliable in mathematical reasoning, with applications in education (step-by-step verification of student work), scientific computing (automated research assistance), and automated problem-solving. Potential risks include over-reliance on external tools and need for robust safety measures in tool call validation.

7 Conclusion

This work demonstrates that language models can effectively learn strategic tool integration through automatic expression conversion and RLOO optimization, achieving 100% tool adoption and doubled accuracy compared to standard approaches. The key insight is that effective tool augmentation requires both proper syntax learning and reinforcement learning optimization of usage patterns, eliminating the need for manual annotation while enabling nuanced tool deployment decisions.

Our framework establishes a foundation for AI systems that augment reasoning through external computational resources rather than relying solely on parametric knowledge. The automatic conversion approach and multi-component reward design provide a template for integrating diverse tools beyond calculators, suggesting broad applicability across reasoning domains.

Future work should explore scaling to complex mathematical tasks, integrating multiple complementary tools within single reasoning chains, and developing sophisticated reward functions that capture reasoning process quality. This research moves toward artificial intelligence that combines human-like reasoning flexibility with superhuman computational accuracy through strategic tool integration.

8 Team Contributions

- **Individual Work:** Designed and implemented the tool-augmented SFT framework including automatic arithmetic expression conversion, RLOO optimization pipeline with tool processing, experimental infrastructure and evaluation pipeline, and analysis with documentation. The final implementation focused specifically on calculator tool integration rather than multiple tool types, and used RLOO based on recent research showing superior sample efficiency.

References

- Asap7772. 2024. Cognitive Behavioral Strategies for Mathematical Reasoning. https://huggingface.co/datasets/Asap7772/cog_behav_all_strategies. Accessed: 2024.
- Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Andrew McCallum, and Diyi Yang. 2023. ReTool: Learning to Use Tools for Mathematical Reasoning. arXiv:2310.03654 [cs.CL]
- Kanishk Gandhi. 2024. Cognitive Behaviors: Countdown Reward Implementation. https://github.com/kanishkg/cognitive-behaviors/blob/main/ver1/Utils/reward_score/countdown.py#L59. Accessed: 2024.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Program-aided Language Models. arXiv:2211.10435 [cs.CL]
- Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. 2023. ReFT: Reasoning with Reinforced Fine-Tuning. arXiv:2401.08967 [cs.CL]
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser

- Kelton, Luke Miller, Maddie Simens, Amanda Askill, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL]
- Jiayi Pan. 2024. TinyZero Countdown Tasks. <https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4>. Accessed: 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLaMA: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789 [cs.AI]
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. RLOO: Reinforcement Learning from Logical Feedback. arXiv:2402.19379 [cs.LG]
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG]
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL]
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL]
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL]